

Project 3: Label Propagation with Apache Giraph

Michael Theophanopoulos

January 2026

1 Implementation

This project implements the Label Propagation algorithm for community detection using Apache Giraph. The algorithm works by having each vertex iteratively adopt the most common label among its neighbors.

1.1 How the Algorithm Works

The basic idea is simple: vertices in the same community should have similar labels. The algorithm proceeds as follows:

1. Start by giving each vertex its own ID as its label
2. In each iteration, look at your neighbors' labels
3. Adopt the most common label (if there's a tie, pick the smallest)
4. Repeat until labels stabilize or max iterations reached

1.2 Key Implementation Details

The compute method handles three main phases:

Initialization (Superstep 0): Each vertex sets its value to its ID and sends it to neighbors.

Label Counting: I use a HashMap to count label frequencies. This is more efficient than scanning the list multiple times.

```
1 Map<Long, Integer> counts = new HashMap<>();
2 for (LongWritable msg : messages) {
3     long label = msg.get();
4     counts.put(label, counts.getOrDefault(label, 0) + 1);
5 }
```

Label Selection: Find the label with highest count. If multiple labels have the same count, pick the smallest one.

Special Cases: When a vertex has only one neighbor, it adopts the neighbor's label only if it's smaller than its current label. This matches the project requirements.

Termination: The algorithm stops when no vertex changes its label, or after 50 iterations.

2 Results

I tested the implementation on the provided input graph (11 vertices, 0-10).

2.1 Graph Structure

The graph has an interesting structure where vertex 10 connects to all other vertices. Here's the adjacency list:

Vertex	Neighbors
0	1, 2, 10
1	0, 3, 4, 5, 10
2	0, 3, 5, 6, 10
3	1, 2, 5, 10
4	1, 5, 10
5	1, 2, 3, 4, 6, 10
6	2, 5, 8, 9, 10
7	8, 9, 10
8	6, 7, 9, 10
9	6, 7, 8, 10
10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9

2.2 Output

The algorithm converged in 5 iterations. All vertices ended up with label 0:

Vertex	Final Label
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0
10	0

2.3 Why This Result Makes Sense

At first glance, having all vertices in one community might seem wrong, but it's actually correct for this graph:

- The graph is strongly connected - you can reach any vertex from any other vertex
- Vertex 10 acts as a hub, connecting all vertices together
- There aren't clear separate clusters

Here's what happens during execution:

- Iteration 1: Everyone sends their own ID
- Iteration 2: Vertex 10 sees labels 0-9, picks 0 (smallest). Other vertices also start seeing 0.

- Iterations 3-5: Label 0 spreads through the network. Since vertex 10 connects everyone, the label propagates quickly.

If the graph had clearer separate communities (like two disconnected groups), we'd see different labels emerge.

3 Code Quality

3.1 Efficiency

Using HashMap for counting is $O(n)$ where n is the number of neighbors. An alternative would be to use `Collections.frequency()`, but that would be $O(n*k)$ where k is the number of distinct labels, since it scans the list for each distinct value.

3.2 Correctness

The implementation handles all the requirements from the project specification:

- Proper initialization
- Majority voting with tie-breaking
- Special case for single neighbor
- Only propagates when labels change
- Terminates correctly

4 Conclusion

The implementation successfully detects communities using label propagation. For the test graph, it correctly identified that all vertices form one community due to the hub structure. The algorithm would work well on graphs with clearer community boundaries.

The key insight is that label propagation is effective when communities are already somewhat separated in the graph structure. When everything is densely connected (like our test case), you get one big community.